SONGA

INTERNAL · PRODUCT & ENGINEERING STRATEGY

Five Revenue Lines on One Codebase

A codebase-honest business proposal for extending Songa's on-demand cargo-trucking marketplace into fleet management, load-pooled ride-share, scheduled booking, B2B corporate cargo, and on-demand goods.

Document	Prepared	Market	Basis
1 of 2 — Platform	**20 July 2026**	**Tanzania · East Africa**	**Live code inventory**

Grounded in a full read of songa_backend_revamp_v2 · songa_admin · driver & rider apps · ws-gateway. Every "reuse" claim is traceable to a file path.

Executive Summary

Songa is already a production-grade freight-trucking marketplace — H3 geo-matching, a full load→transit→unload trip lifecycle, live GPS tracking, and a tamper-evident finance ledger are all shipped. The five business lines below are **not rewrites**; they are adapters and new surfaces on top of an engine that already exists. This document ranks them by how much of that engine each one can reuse.

CORE ENGINE
~80%
of trip, matching, tracking & ledger logic is reusable across all 5 lines

FASTEST WIN
#3
Scheduling — mostly wiring two existing stub screens + a Celery-beat job

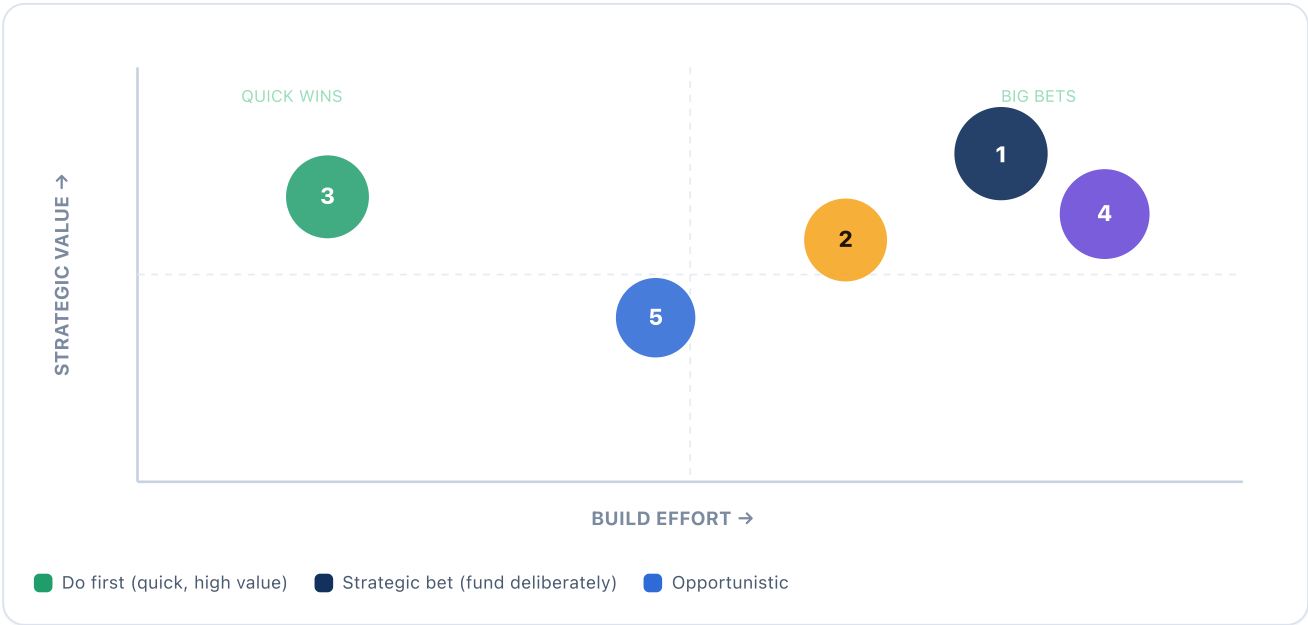
BIGGEST LEVER
#1
Fleet ownership — unlocks supply scaling beyond 1 driver = 1 truck

NEW MUSCLE
#4 · #5
Corporate billing & a goods/vendor catalog are true greenfield

The five lines at a glance

#	BUSINESS LINE	ONE-LINER	REUSE	BUILD EFFORT	STRATEGIC VALUE
1	**Fleet Management**	Multi-truck owners assign drivers & get consolidated reports	HIGH	L	★★★★★
2	**Cargo-weight Ride-Share**	Pool loads heading the same way by remaining capacity	MED	M–L	★★★★
3	**Scheduled Booking**	Book-for-later & recurring haul contracts	HIGH	S–M	★★★★
4	**B2B / Corporate Cargo**	Companies request trucks for multi-stop delivery jobs	MED	L	★★★★★
5	**Goods Selling**	Order a tipper of sand / tanker of water — good + haulage	MED	M–L	★★★★

Effort vs. strategic value



Recommended sequence: ship #3 Scheduling first (unblocks demand shaping and is nearly free), stand up #1 Fleet Ownership as the platform's supply-side backbone, then layer #4 Corporate on top of both. #2 and #5 are demand experiments that plug into the same rails once #1/#3/#4 exist.

The Reuse Surface — What Already Exists

Before the individual cases, here is the shared engine every proposal draws on. This is why these are extensions, not new products.

CAPABILITY	STATUS	WHERE IT LIVES	FEEDS LINES
Cargo trip lifecycle (REQUESTED→ACCEPTED→LOADING→EN_ROUTE→UNLOADING→COMPLETED)	EXISTS	`trips/models/Trip.py`, `trips/tasks/trip_operations.py`	All
Cargo type + tonnage brackets (350kg–60t), CarType↔CargoType allow-lists	EXISTS	`vehicles/models/`, `data/size_choices.dart`	1,2,5
H3 geo-matching, tunable dispatch waves, multi-factor scoring	EXISTS	`config/matching.py`, `utils/raw/driver_matching.py`	2,3,4
Live GPS tracking → partitioned breadcrumbs → polyline/traffic analytics	EXISTS	`TripLocationHistory`, `TripActivitySummary`, ws-gateway	1,4 (Do core)
Hash-chained double-entry ledger, DriverWallet (balance/debt), settlements, 10% service charge, invoices	EXISTS	`payments/models/`, `payments/utils/payment_processing.py`	All
KYC vetting workflow; courier/helper (loaders) fields on trip	EXISTS	`admin_api/views/kyc.py`, `Trip.number_of_couriers`	1,4,5
RBAC admin — Next.js 16 + deck.gl H3 heatmaps, analytics funnel/revenue/geography	EXISTS	`songa_admin/…/dashboard/*`, `admin_api/`	1,4 dashbo
Real-time bus — Kafka → Java Spring WebFlux WS gateway; Google Maps; NextSMS	EXISTS	`utils/kafka.py`, `services/`, `songa_ws_gateway/`	All (notify/
Multi-vehicle ownership (one owner → many trucks, driver assignment)	MISSING	`Driver.vehicle` is OneToOne — no `FleetOwner`	Build fo
Scheduled / recurring / book-for-later trips	MISSING	Immediate-only; "Scheduled Rides" screens are stubs	Build fo
Corporate / B2B account & billing entity	MISSING	Only internal `company_account` for service charge	Build fo
Goods marketplace / product catalog / vendor listings	MISSING	No product/listing/order models	Build fo
Live payment gateway (M-Pesa/Airtel/Tigo/card)	PARTIAL	Labels + placeholder tokens only; cash-settled today	Cross-cutting
Numeric cargo-weight input by rider; load pooling/consolidation	MISSING	Rider picks vehicle by capacity, not numeric weight	Build fo

REUSABLE ENGINE
8
shipped subsystems shared across the lines

NET-NEW TO BUILD
5
entities/surfaces (fleet, schedule, corporate, goods, tracking-API)

CROSS-CUTTING
2
payment gateway + push notifications benefit every line

Cross-cutting note: a real mobile-money/card gateway and push notifications are *not* owned by any single line but multiply the value of all five. Treat them as a shared platform track funded alongside line #3.

1 **Fleet Management for Multi-Vehicle Owners**
Owner accounts · driver assignment · consolidated earnings & utilisation reports
Effort L

2 **Cargo-Weight Ride-Share & Load Consolidation**
Numeric weight · remaining-capacity pooling · split-fare ledger
Effort M–L

3 **Ride Booking & Scheduling**
Book-for-later · recurring contracts · pre-dispatch scheduler
Effort S–M

4 **B2B / Corporate Cargo Platform**
Company accounts · credit terms · multi-stop delivery jobs
Effort L

5 **Goods Selling — Materials on Demand**
Sand / rock / water catalog · vendor payouts · order→trip bridge
Effort M–L

How to read each chapter

Every business line follows the same eight-part template so you can compare them apples-to-apples: **Concept → Maps to existing code (Reuse) → What to build (Gap) → Data-model changes → Revenue model → MVP scope & effort → Risks → Roadmap slot.** Green pills mean it already ships; blue means net-new work; the split panel shows reuse-vs-build side by side.

1 Fleet Management for Multi-Vehicle Owners

Effort L · Value ★★★★★

Today one driver equals one truck (`Driver.vehicle` is a OneToOne). But real supply in Tanzanian trucking is owned by **fleet owners** — a business with 3–40 trucks and hired drivers. Give owners an account to register many vehicles, assign drivers, and receive consolidated earnings, utilisation and settlement reports. This is the single highest-leverage change because it unlocks supply scaling.

Concept

- A **Fleet Owner** registers, passes KYC once, then adds N vehicles (reusing the existing vehicle registration + document flow).
- Owner invites/assigns drivers to specific trucks; a driver can be swapped between vehicles.
- Owner portal shows per-vehicle and fleet-wide **earnings, trips, utilisation, idle time, debt/settlement status** — all data the platform already computes per trip.
- Settlements roll up to the owner's wallet (owner is the money counterparty), with per-driver visibility.

Reuse vs. Build

✓ REUSE (ALREADY SHIPS)

- Vehicle registration + document upload & KYC watermarking — `vehicles/models/`, `upload_documents_page.dart`
- Per-trip earnings, service charge, `DriverWallet` balance/debt — `payments/`
- Settlement / payout workflow — `admin_api` finance `settlements-queue`
- Trip & utilisation analytics (trips, revenue, geography) — `admin_api/views/analytics`
- Live vehicle positions & H3 supply density — `live-map`, deck.gl

▲ BUILD (NET-NEW)

- `FleetOwner` entity (OneToOne User) + owner KYC state
- Break `Driver.vehicle` OneToOne → `Vehicle.owner` FK + `DriverAssignment` (driver↔vehicle, effective dates)
- Owner web/mobile portal (owner-scoped, RBAC role `FLEET_OWNER`)
- Owner-scoped report endpoints & PDF/CSV export
- Owner wallet as settlement counterparty + per-driver sub-ledger

Data-model changes

CHANGE	TYPE	NOTES
`FleetOwner(user, business_name, tin, kyc_status)`	New model	Mirror `Driver` KYC pattern
`Vehicle.owner → FK(FleetOwner)`	Alter	Nullable for solo drivers (back-compat)
`DriverAssignment(driver, vehicle, active_from, active_to)`	New model	Replaces implicit OneToOne; trip records which assignment was live
`OwnerWallet` or reuse `Account` typing	Reuse+alter	Settlement destination = owner when vehicle is fleet-owned

Migration care: the OneToOne→many change touches dispatch (which driver is on which truck now) and settlement (who gets paid). Keep solo drivers working by treating "no owner" as self-owned. This is the reason for the **L** rating despite heavy reuse.

Revenue model

PRIMARY
Take-rate
Same 10% service charge, now at fleet scale — more trucks online

SAAS TIER
Fleet+
Monthly per-vehicle fee for advanced reports, maintenance logs, fuel tracking

FINANCIAL
Float
Settlement float & future asset-financing referral fees

MVP scope & effort

- **MVP:** owner account + KYC, add multiple vehicles, assign one driver per vehicle, consolidated earnings + utilisation report, owner wallet settlement. ~6–8 wks
- **Fast-follow:** driver swapping, per-vehicle maintenance/fuel logs, CSV/PDF export, owner mobile app.

Risks: (1) OneToOne→many migration risk on live dispatch/settlement — mitigate with a shadow read + backfill. (2) Owner-vs-driver payout disputes — the hash-chained ledger already gives an audit trail; expose it in the owner portal.

2 Cargo-Weight Ride-Share & Load Consolidation

Effort M–L · Value ★★★★

A 10-tonne truck carrying a 3-tonne load has 7 tonnes of wasted capacity heading in a direction someone else needs. Let shippers whose cargo is going the same way **share a truck**, matched by remaining capacity and route overlap, and split the fare. This raises truck utilisation and lowers per-shipper price — the classic pooling play, applied to freight.

Concept

- Shipper enters a **numeric cargo weight/volume** (new) instead of only picking a vehicle by capacity.
- Matching engine finds trucks already routed along a corridor with **remaining capacity ≥ this load** and acceptable detour.
- Multi-pickup / multi-drop trip; fare split proportionally across shippers in the shared ledger.
- Opt-in: "share to save" vs. "book exclusive" toggle at booking.

Reuse vs. Build

✓ REUSE

- H3 corridor matching + dispatch waves — `config/matching.py`, `driver_matching.py`
- Tonnage brackets & CarType max-load / capacity — `vehicles/models/`, `size_choices.dart`
- Courier/loader fields for multi-stop labour — `Trip.number_of_couriers`
- Ledger for splitting a fare across payers — `payments/utils/payment_processing.py`
- Live tracking of a multi-leg route — `TripActivitySummary` polylines

▲ BUILD

- Numeric weight/volume input in rider app (replace capacity-only picker)
- `remaining_capacity` tracking on an in-progress trip
- Multi-shipper trip model: N pickups / N drops, per-leg sequencing
- Route-overlap + detour-tolerance scoring extension to matcher
- Proportional **split-fare** ledger entries + per-shipper invoices

Data-model changes

CHANGE	TYPE	NOTES
`Trip.declared_weight_kg`, `declared_volume`	Add	Validate against `vehicle_max_load_capacity`
`SharedTrip` / `TripLeg(trip, shipper, pickup, drop, seq, weight)`	New	One truck-run, many shipper legs
`Trip.remaining_capacity_kg`	Add (derived)	Decrements as legs load
Split `PaymentRecord` per leg	Extend	Sum of legs = truck-run fare; each shipper invoiced their share

Revenue model

UTILISATION
↑ trips/truck
More paid km per truck = more take-rate volume

PRICING
Pool premium
Platform keeps a slice of the shipper's savings vs. exclusive

RETENTION
Lower price
Cheaper shared rate wins price-sensitive SME shippers

MVP scope & effort

- **MVP:** numeric weight input + 2-shipper same-corridor pairing + split fare. Start with **manual/assisted matching** before full auto-consolidation. ~7–9 wks
- **Fast-follow:** N-shipper consolidation, backhaul matching (return-leg loads), detour-tolerance tuning.

> **Honest risk:** full auto-consolidation/backhaul is a genuinely hard optimisation problem and liability-sensitive (whose cargo is delayed by whose detour?). De-risk by launching 2-party, same-direction pooling with tight detour caps before generalising. This is why it is **M–L**, not **M**.

3 Ride Booking & Scheduling

Effort S–M · Value ★★★★

Freight is often planned, not spontaneous — "collect at 6am tomorrow", "every Monday to the warehouse". The apps already ship **stub screens** for this (driver "Scheduled Rides", rider calendar tab), and the entire trip lifecycle already works. Scheduling is mostly a **time trigger + wiring existing stubs** — the fastest, cheapest win in this document.

Concept

- Shipper picks a future date/time (or a recurrence rule) at booking.
- A scheduler pre-dispatches the trip ahead of pickup time using the existing matching engine.
- Reminders to shipper & driver via the existing WS/SMS channels (push as fast-follow).
- Recurring "contracts" for repeat lanes (daily/weekly) — the on-ramp to corporate (line #4).

Reuse vs. Build

✓ REUSE (THIS IS WHY IT'S CHEAP)

- Complete trip lifecycle & dispatch — `trips/tasks/trip_operations.py`
- Celery + Celery-beat already in the stack — `songaServer/celery.py`
- Stub UI already placed: driver "Scheduled Rides" (`driver_drawer.dart`) + rider calendar tab
- Pricing/estimate engine — `pricing/`
- WS/SMS reminders — `utils/kafka.py` , `services/nextSMS.py`

▲ BUILD

- `Trip.scheduled_for` + status `SCHEDULED`
- `RecurrenceRule` (RRULE-style) + generator job
- Celery-beat pre-dispatch task (lead-time aware)
- Flesh out the two stub screens (date/time picker, list, cancel/edit)
- Reminder scheduling + no-driver-found fallback re-dispatch

Data-model changes

CHANGE	TYPE	NOTES
`Trip.scheduled_for (datetime, null)` + `SCHEDULED` status	Add	Null = immediate (today's behaviour, back-compat)
`RecurrenceRule(rider, template, rrule, next_run)`	New	Spawns child trips
Celery-beat: `dispatch_scheduled_trips`	New task	Runs each minute; dispatches when now ≥ scheduled_for – lead

Revenue model

DEMAND SHAPING
Fill troughs
Pre-booked trips smooth supply, cut no-driver-found

CONVENIENCE FEE
Priority
Small premium for guaranteed scheduled pickup

STICKINESS
Contracts
Recurring lanes lock in repeat shippers

MVP scope & effort

- **MVP:** single future-dated booking, scheduler pre-dispatch, reminders, wired stub screens. ~3–4 wks
- **Fast-follow:** recurrence rules, priority guarantee, driver-side accept-ahead.

> **Why do this first:** smallest effort, unblocks demand shaping, reduces the No-Driver-Found funnel loss the admin analytics already surfaces, and it is the natural precursor to corporate contracts (#4).

4 B2B / Corporate Cargo Platform

Effort L · Value ★★★★★

Hardware stores, furniture and electronics retailers, distributors — they don't want a consumer app, they want a **company account** that requests trucks, batches many deliveries into one dispatched job, and pays on monthly invoice/credit terms. This is the highest-ARPU segment and reuses the whole delivery engine; the new muscle is **account, credit & multi-stop job** modelling.

Concept

- A **Corporate Account** with multiple staff members (roles: requester, approver, finance).
- Staff create a **delivery job** = many drop-offs (customer orders) assigned to one or more trucks.
- Billing on **credit terms + consolidated monthly invoice** (reusing the invoice/ledger system), not per-trip cash.
- Corporate dashboard: live tracking of all active deliveries, proof-of-delivery, spend reports.

Reuse vs. Build

✓ REUSE

- Trip lifecycle + courier/loaders for furniture/appliances — `Trip`, `number_of_couriers`
- Invoice + hash-chained ledger for monthly billing — `payments/models/Invoice.py`
- Admin RBAC pattern for corp member roles — `admin_api/roles.py`
- Live tracking + ETA for the delivery dashboard — ws-gateway, `TripActivitySummary`
- Pricing brackets for negotiated corporate rates — `pricing/`

▲ BUILD

- `CorporateAccount` + `CorporateMember` (roles) + invite flow
- `CreditTerms` (limit, net-days) + monthly invoice run
- `DeliveryJob` wrapping N trips (multi-stop, one or many trucks)
- Corporate web portal (bulk upload orders, approve, track, download invoices)
- Proof-of-delivery capture (signature/photo) per drop

Data-model changes

CHANGE	TYPE	NOTES
`CorporateAccount(name, tin, credit_limit, net_days)`	New	Billing counterparty
`CorporateMember(account, user, role)`	New	requester / approver / finance
`DeliveryJob(account, created_by, status)` + `Trip.job → FK`	New + alter	One job → many trips
Monthly invoice run over a job/account	Extend	Reuse `Invoice` subtotal/VAT/total
`ProofOfDelivery(trip, signature, photo, ts)`	New	Uses existing S3 storage

Revenue model

VOLUME + ARPU
Highest
Corporates ship daily; predictable recurring GMV

TERMS PREMIUM
Credit
Net-30 billing + platform fee; optional financing spread

CONTRACTS
Lock-in
SLA + negotiated lane pricing = defensible relationships

MVP scope & effort

- **MVP:** corporate account + members, a multi-stop delivery job, live tracking dashboard, monthly consolidated invoice. ~8–10 wks
- **Fast-follow:** credit-limit enforcement, bulk CSV order upload, proof-of-delivery, API for integration with their ERP.

Risks: credit risk (net-30 exposure) needs limits + suspension logic; multi-stop routing/sequencing is non-trivial — start with manual stop ordering. Depends on #1 (fleet supply) and benefits from #3 (scheduling).

5 Goods Selling — Materials on Demand

Effort M–L · Value ★★★★

"Order a tipper of sand", "a tanker of water", "a load of aggregate to the site" — construction and household demand where **the price bundles the material and its haulage**. Songa already models Tipper and Tanker car-types and owns the delivery leg; the new muscle is a **productised catalog with vendors and a vendor-payout split**. This turns Songa from a pure carrier into a merchant.

Concept

- Customer browses a catalog of goods (sand, rock/aggregate, water, cement, gravel) sold by unit/truckload.
- Order = material price + haulage; Songa dispatches an appropriate truck-type to the supply point then the site.
- Vendors (quarries, water depots) fulfil; ledger splits payment between vendor, driver/owner, and platform.
- Live tracking of the delivery reuses the standard trip experience.

Reuse vs. Build

✓ REUSE

- CarType Tipper / Tanker already modelled — `vehicles/models/CarType`
- Pricing brackets for the haulage component — `pricing/`
- Trip lifecycle load→transit→unload = material delivery — `Trip`
- Ledger for multi-party split (vendor + carrier + platform) — `payments/`
- Live tracking + ETA to the site — ws-gateway

▲ BUILD

- `GoodsListing` / product catalog + units & pricing
- `Vendor` entity + supply-point locations + stock/availability
- `GoodsOrder` → bridges to a dispatched `Trip`
- Vendor payout split in the ledger + vendor settlement
- Customer catalog UI (browse, order, reorder)

Data-model changes

CHANGE	TYPE	NOTES
`Vendor(name, supply_points, payout_account)`	New	Quarry / water depot / builder's merchant
`GoodsListing(vendor, name, unit, unit_price, car_type)`	New	Links a product to the truck-type that carries it
`GoodsOrder(customer, listing, qty, site) → Trip`	New + bridge	Order spawns a standard trip from supply-point → site
3-way split `LedgerEntry` (vendor / carrier / platform)	Extend	Vendor settlement reuses `Settlement` pattern

Revenue model

MERCHANT MARGIN
Goods spread
Margin on material + the usual haulage take-rate

BASKET SIZE
Higher
A truckload of material dwarfs a plain delivery fare

VENDOR SAAS
Listings
Fee/subscription for vendors to reach demand

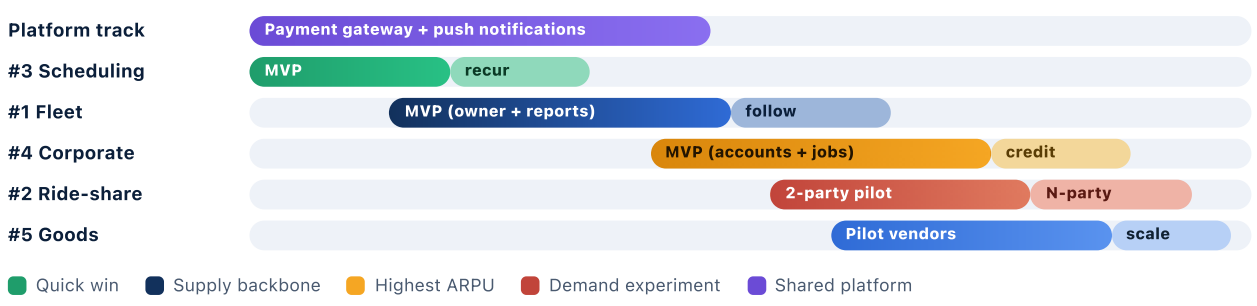
MVP scope & effort

- **MVP:** 3–5 material listings from 1–2 pilot vendors, order→trip bridge, 3-way payout split, catalog UI. ~7–9 wks
- **Fast-follow:** vendor self-serve listings, stock/availability, reorder, quality/quantity dispute handling.

Risks: inventory & quality liability (short-loaded sand, water quality) — Songa becomes accountable for the good, not just the ride. Start consignment-style with vetted vendors and clear dispute rules. Vendor payment reconciliation adds ledger complexity.

Sequenced Roadmap

Ordered by dependency and value-per-effort. Bars are indicative build windows, not commitments; a shared "platform track" (payment gateway + push) runs alongside.



Dependency logic

- **#3 first** — cheapest, unblocks demand shaping, precursor to recurring corporate lanes.
- **#1 next** — the supply backbone; corporate volume needs many trucks under management.
- **#4 on top of #1+#3** — corporate multi-stop jobs lean on fleet supply and scheduling.
- **#2 & #5 in parallel as experiments** — plug into the same trip/ledger rails once the backbone exists.
- **Platform track throughout** — a live mobile-money/card gateway and push notifications lift every line and remove the cash-only ceiling.

What this document is (and isn't)

It is a codebase-honest map: every "Exists / Reuse" claim above is traceable to a real file in `songa_backend_revamp_v2`, `songa_admin`, or the Flutter apps, and every "Build" item is a confirmed gap (no matching model/endpoint found).

It isn't a market-sizing or financial-forecast deck — effort ratings are engineering-relative (S/M/L), and week ranges are directional. Firm estimates need a spike per line against the current schema.